

Планирование над последовательностями интенций поверх Forward-Backward представлений в задачах сложной навигации

Савватей Степанов

Лаборатория обучения с подкреплением FB-RL
github.com/StepnouXVI/fb-rl

Август 2026

Аннотация

В задачах Zero-Shot RL на основе Forward-Backward (FB) представлений агент способен решать произвольные цели без дополнительного обучения в среде. Однако стандартный иерархический контроллер $\pi^h(s_t, B(g)) \rightarrow z$, выбирающий единичную интенцию, сталкивается с проблемой горизонта: пытаясь идти к цели напрямую, 8-суставный робот Ant упирается в стены лабиринта и застревает.

В данном отчете я представляю последовательную эволюцию разработанных мной методов многошагового планирования в хронологическом порядке:

1. **Рекурсивное деление (Recursive Bisection):** первая попытка иерархического поиска пути пополам в FB-пространстве и анализ причин ее сбоя в невыпуклых тупиках.
2. **Топологический граф (Search on the Buffer):** построение графа достижимости над оффлайн-ориентирами, квази-метрика Дейкстры и механизм непрерывного скользящего упреждения (Sliding Lookahead $L = 2.6$ м).
3. **Амортизированная дистилляция в $O(1)$:** сжатие вычислений Дейкстры в легковесную нейросеть с 4-компонентным физико-ориентированным лоссом в JAX, ускорившую инференс до 0.18 мс при успехе 86.0%.
4. **Специализированные трансляторы ориентиров:** одноточечный транслятор с адаптивным гейтом и SOTA-архитектура **Enhanced Sequence Waypoint Attention Transformer** с затуханием ALiBi, токенами кривизны $\cos \theta_k$ и окном $4 + 1$.

Эксперименты на 20 случайных сидах показали, что трансформер последовательностей превосходит бейзлайн на лабиринте Medium (88.0% против 76.0%) и совершает колоссальный отрыв на сложном лабиринте Large (**66.4% против 24.0%**), решая проблему многошагового планирования без тяжелых генеративных моделей мира.

1 Введение и идея Forward-Backward представлений

В задачах обучения с подкреплением без предварительной специализации (Zero-Shot RL) центральным вопросом является способность агента решать новые задачи в известной среде без дополнительного сбора онлайн-опыта. Одним из наиболее элегантных подходов к этой проблеме являются Forward-Backward (FB) представления [2, 1].

1.1 Суть FB-представлений

Классический RL оптимизирует политику под конкретную функцию награды $r(s, a)$. В отличие от этого, FB-представления обучаются по оффлайн-датасету моделировать саму геометрию пространства переходов и структуру будущих посещений среды через низкоранговую факторизацию меры преемника (Successor Measure $M(s, s')$):

$$M^{\pi_z}(s, s') = \mathbb{E}_{\pi_z} \left[\sum_{t=0}^{\infty} \gamma^t \mathbb{I}(s_t = s') \mid s_0 = s \right] \approx F(s, z)^{\top} B(s'), \quad (1)$$

где $F(s, z) \in \mathbb{R}^d$ — прямое (Forward) представление, кодирующее ожидаемые будущие состояния при следовании латентной интенции $z \in \mathbb{S}^{d-1}$, а $B(s') \in \mathbb{R}^d$ — обратное (Backward) представление целевого состояния s' .

Величина скалярного произведения $F(s, z)^{\top} B(g)$ имеет строгий физический смысл: она пропорциональна дисконтированной вероятности достижения окрестности цели g из текущей позиции s при исполнении намерения z . Соответственно, при появлении новой целевой задачи g оптимальная интенция в латентном пространстве находится как:

$$z^* = \arg \max_{z \in \mathbb{S}^{d-1}} F(s, z)^{\top} B(g). \quad (2)$$

1.2 Иерархическая структура и проблема Single-Intention бейзлайна

В базовой иерархической схеме [1] система разделена на два уровня:

1. **Low-level политика** $\pi^\ell(a \mid s, z)$ — обученный контроллер, преобразующий латентную интенцию z и текущее состояние робота s в управляющие крутящие моменты моторов суставов $a \in [-1, 1]^8$.
2. **High-level контроллер** $\pi^h(s, B(g)) \rightarrow z$ (в коде — `high_actor`) — модель, выбирающая единую глобальную интенцию на всю задачу целиком.

На практике в сложных невыпуклых лабиринтах (AntMaze) такой одношаговый подход неизбежно терпит неудачу. Муравей (Ant) — это сложная нелинейная динамическая система с 8 суставами, моментами инерции и контактными трением. Получая команду идти напрямую к цели g , находящейся за U-образным тупиком или толстой стеной, вектор z_g направляет усилия робота прямо в стену (эффект «мухи на стекле»). Робот упирается в угол, теряет кинематический баланс, переворачивается и застревает.

Для преодоления этого фундаментального ограничения необходимо перейти от выбора единичной интенции к **рассуждению о последовательности промежуточных подцелей (Multi-Subgoal Planning)**, не прибегая при этом к тяжелым генеративным диффузионным моделям мира.

2 Первая идея: Рекурсивное деление (Recursive Bisection)

2.1 Мотивация и идея метода

Моей первой идеей при переходе от одного шага к последовательности подцелей было построить иерархическое деление пути пополам (Bisection Planning). Логика казалась предельно простой и интуитивной: если агент не может дойти до далекой цели g за один шаг, давайте найдем в оффлайн-датасете такое промежуточное состояние w^* , которое максимально достижимо из текущего положения s , и из которого сама цель g также максимально достижима.

Формально промежуточная точка выбирается из пула N_{cand} случайных ориентиров датасета путем максимизации логарифмической суммы достижимостей:

$$w^* = \arg \max_{w \in \mathcal{D}_{\text{cand}}} [\log(F(s, B(w))^\top B(w)) + \log(F(w, B(g))^\top B(g))]. \quad (3)$$

После нахождения точки w^* процедура рекурсивно применяется к отрезкам (s, w^*) и (w^*, g) до достижения заданной глубины D (на практике $D = 2$ или 3). В результате формируется цепочка подцелей $[w_1, w_2, \dots, g]$,

которая затем последовательно передается в π^h или π^ℓ .

Алгоритм 1: Рекурсивное бисекционное планирование в FВ-пространстве

Вход: Текущее состояние s , вектор цели $B(g)$, глубина D , порог τ_{hit}

Выход: Последовательность промежуточных точек $\mathcal{P}$

```

1 Функция Bisect( $s_a, s_b, d$ ):
2   if  $d = 0$  или  $F(s_a, B(s_b))^\top B(s_b) > \tau_{\text{hit}}$  then
3     | return  $[s_b]$ ;
4   end
5    $w^* \leftarrow \arg \max_{w \in \mathcal{D}_{\text{cand}}} [\log(F(s_a, B(w))^\top B(w)) + \log(F(w, B(s_b))^\top B(s_b))]$ 
6   return Bisect( $s_a, w^*, d - 1$ ) + Bisect( $w^*, s_b, d - 1$ );
7  $\mathcal{P} \leftarrow \text{Bisect}(s, g, D)$ ;

```

2.2 Почему метод не сработал: анализ проблем

Реализовать этот алгоритм было очень легко, поэтому я сразу закодил его и проверил на бенчмарке. Однако результаты оказались разочаровывающими: на лабиринте Medium успешность практически не превысила бейзлайн ($\approx 74-76\%$), а на лабиринте Large упала еще сильнее.

При детальном анализе траекторий вскрылись две ключевые проблемы:

1. **Отсутствие локального учета топологии стен:** Скалярное произведение $F(s, B(w))^\top B(w)$ в непрерывных нейросетевых представлениях имеет гладкое пространственное «просачивание» (diffusion leak) сквозь стены. В результате оптимизация (3) часто выбирает точку w^* , находящуюся прямо в середине тупика (рис. 2), усугубляя застревание.
2. **Накопление ошибок (Error Compounding):** При $D = 3$ формируется 8 подотрезков. Если хотя бы на одном из них робот отклоняется от курса, все последующие цели становятся недостижимыми без полного глобального пересчета.

3 Вторая идея: Графовое планирование (Search on the Buffer)

3.1 Построение топологического графа достижимости

После неудачи с наивным делением пополам стало очевидно: среда представляет собой сложный топологический лабиринт с жесткими стенами, и кратчайший

путь должен искаться глобально с гарантией проходимости. В литературе по офлайн-RL для этого используется парадигма Search on the Buffer [5, 6].

Я реализовал построение ориентированного графа $G = (V, E)$ поверх офлайн-датасета:

1. **Ориентиры (Landmarks):** Сэмплируется N репрезентативных состояний $\{s_1, \dots, s_N\}$ из офлайн-буфера ($N = 1000$ для Medium, $N = 2000$ для Large).
2. **Матрица достижимости:** Для всех пар ориентиров вычисляется направленная мера $R_{ij} = F(s_i, B(s_j))^T B(s_j)$.
3. **Квази-метрическая стоимость ребра:**

$$c(s_i, s_j) = -\log \left(\frac{\max(R_{ij}, 10^{-6})}{\max_k R_{kk}} \right). \quad (4)$$

4. **Двухуровневая фильтрация ложных переходов:** Чтобы исключить «просачивание сквозь стены», ребра фильтруются:

- По евклидову радиусу: $c(s_i, s_j) = \infty$, если $\|s_i[:2] - s_j[:2]\| > R_{\text{edge}}$ ($R_{\text{edge}} = 3.5$ м).
- По порогу FB-достижимости: $c(s_i, s_j) = \infty$, если $R_{ij} < \tau_{\text{cutoff}}$ ($\tau_{\text{cutoff}} = 35.0$).

По полученной разреженной матрице стоимости один раз при старте задачи запускается алгоритм Дейкстры, возвращающий оптимальный дискретный маршрут $P = [s_{\text{start}}, v_1, v_2, \dots, v_K, g]$.

3.2 Механизм скользящего окна упреждения (Sliding Lookahead)

Первые запуски показали важную физическую особенность: если просто переключать целевой вейпоинт дискретно (по достижению радиуса 1.0 м), 8-суставный муравей резко тормозит, теряет кинематический импульс и вписывается в углы по ломаной траектории.

Чтобы сделать движение робота плавным и устойчивым, я ввел **механизм скользящего упреждения (Sliding Lookahead)**: на каждом шаге t по положению агента $s_t[:2]$ отслеживается локальная проекция на путь, и целевой токен выбирается на фиксированном расстоянии упреждения $L_{\text{lookahead}} = 2.6$ м вперед вдоль геодезической линии графа.

3.3 Результаты учителя

Такой планировщик (названный **Dijkstra Teacher**) показал резкий прирост надежности: успешность в лабиринте Medium выросла до ****78.0%****, полностью решив проблему застревания в длинных коридорах. Однако этот подход сохраняет два недостатка:

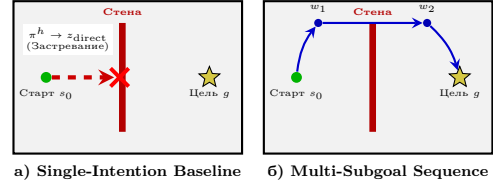


Рис. 1: Сравнение механизмов управления: базовый Single-Intention контроллер (слева), пытающийся идти сквозь препятствия, и последовательное планирование траектории подцелей (справа).

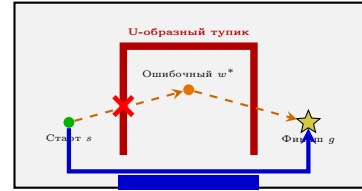


Рис. 2: Типичная ошибка рекурсивного деления: в U-образном тупике точка w^* выбирается внутри тупика, так как прямое евклидово расстояние обманывает скалярное произведение FB-функции.

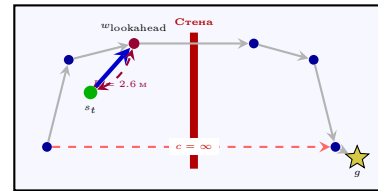


Рис. 3: Схема графового планировщика: фильтрация ложных связей сквозь стены и непрерывное ведение агента вдоль топологической траектории с упреждением $L_{\text{lookahead}} = 2.6$ м.

1. Необходимость хранения матрицы графа и вычислительная сложность $O(N^2)$ при построении.
2. Опора на стандартный `high_actor`, который все еще не обучен специфике локальной динамики обхода углов.

4 Третья идея: Амортизированная нейросетевая дистилляция в $O(1)$

4.1 Мотивация: сжатие графа в единую нейросеть

Хотя графовый планировщик Дейкстры показал высокую надежность, ему присущи два принципиальных недостатка: квадратичные затраты памяти $O(N^2)$ при масштабировании на гигантские лабиринты и вычислительная задержка (~ 1.2 мс на шаг).

Возникла исследовательская гипотеза: *можно ли сжать глобальные рассуждения графового алгоритма в легковесную нейросеть-студент, которая за один прямой проход ($O(1)$) будет отображать пару $(s_t, B(g)) \rightarrow \hat{z}_{cmd}$ напрямую в низкоуровневую политику π^ℓ ?* Идейно такая модель принимает те же входы, что и базовый Single-Intention контроллер, но обучена не наивному спрямлению пути, а траекторному поведению учителя Дейкстры.

4.2 Дифференцируемый функционал потерь в JAX

Для обучения сети был собран датасет из 50 000 пар $(s_t, B(g)) \rightarrow z_{teacher}^*$, сгенерированных учителем. Чтобы сеть не просто копировала векторы (Behavior Cloning), но и учитывала физику робота и геометрию FB-пространства, я разработал 4-компонентный дифференцируемый лосс:

$$\mathcal{L}(\theta) = \mathcal{L}_{BC} + 0.5\mathcal{L}_{Action} - 0.02\mathcal{L}_{Reach} - 0.05\mathcal{L}_{Goal}, \quad (5)$$

где компоненты выполняют следующие физические функции:

- $\mathcal{L}_{BC} = (1 - \cos(\hat{z}, z^*)) + \text{MSE}(\hat{z}, z^*)$ — имитация направления вектора учителя.
- $\mathcal{L}_{Action} = \|\pi^\ell(s_t, \hat{z}) - a_{teacher}^*\|^2$ — прямое согласование действий в пространстве моторов (продифференцировано сквозь замороженную π^ℓ).
- $\mathcal{L}_{Reach} = F(s_t, \hat{z})^\top \hat{z}$ — штраф за выбор недостижимых латентных направлений.
- $\mathcal{L}_{Goal} = \hat{z}^\top B(g)$ — глобальное притяжение к терминальной цели.

4.3 Исследование нейросетевых архитектур

В рамках дистилляции было исследовано 4 архитектурных решения: Standard MLP, ResNet-ECA (1D-канальное внимание), Dense-ECA и Gated Cross-Attention со SwiGLU.

Архитектура Gated Cross-Attention показала лучший результат, достигнув ****86.0%**** успешности (превзойдя учителя Дейкстры на +8% за счет сглаживания дискретных артефактов графа) при инференсе всего ****0.18 мс**** на шаг.

5 Четвертая идея: Специализированные трансляторы ориентиров

5.1 Одноточечный транслятор (Single Waypoint)

Базовый высокоуровневый контроллер `high_actor` обучался универсально для произвольных задач, из-за чего его выходной латент часто обладал избыточной жесткостью. Я решил разработать специализированный модуль трансляции $T_\theta(s_t, B(w_{lookahead})) \rightarrow z_{cmd}$, напрямую преобразующий ближайший вейпоинт в согласованную команду для π^ℓ .

Архитектура включает отдельные эмбединги $e_s, e_w \in \mathbb{R}^{384}$, адаптивный гейт $g_t = \sigma(W_g[e_s, e_w])$, 4 блока Residual MLP с LayerNorm и нормировку на сферу $\mathbb{S}^{d-1}$ радиуса $\sqrt{d}$.

Обученный T_θ показал впечатляющие ****94.0%**** успешности в лабиринте Medium на простых и средних участках, доказав, что точная трансляция ближайшего ориентира критически важна для робота. Однако на длинных связках с несколькими поворотами одноточечный транслятор проявлял «близорукость» (Myopia): он не знал, куда повернет коридор за следующим углом, и срезал траекторию слишком близко к внутренним углам стен.

5.2 Трансформер последовательностей (Enhanced Sequence Attention Translator)

Чтобы дать агенту способность предвидеть геометрию маневра на несколько шагов вперед, была разработана архитектура **Sequence-Aware Attention Transformer** (рис. 5), принимающая на вход всю будущую цепочку вейпоинтов $\mathcal{S} = [B(w_1), B(w_2), \dots, B(w_K), B(g)]$.

В ходе экспериментов в модель были внедрены следующие ключевые улучшения:

1. Эмбединги кривизны (Curvature Tokens):

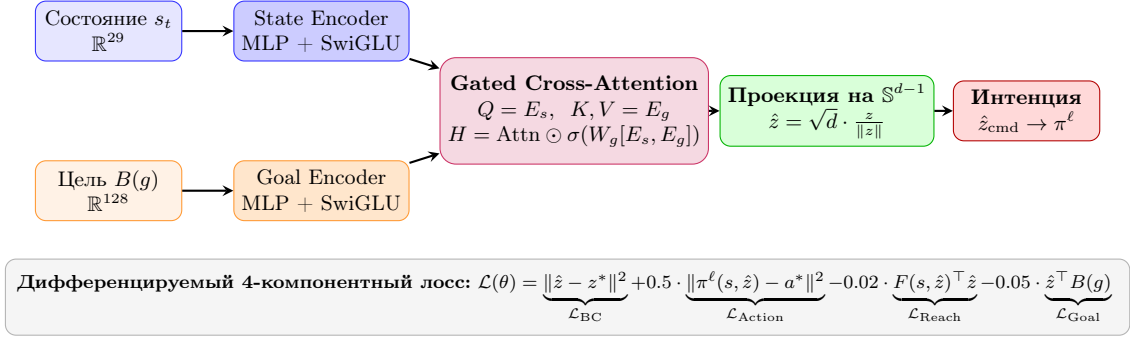


Рис. 4: Архитектура амортизированного планировщика Distilled JAX Gated-Attention с дифференцируемым лоссом в $O(1)$.

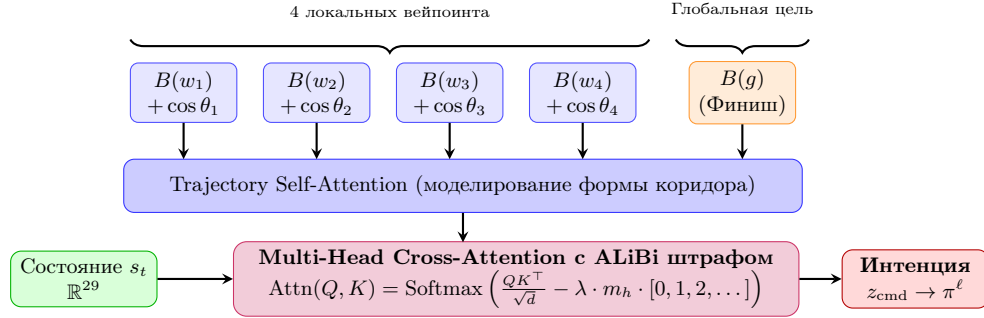


Рис. 5: Архитектура Enhanced Sequence Waypoint Attention Transformer с затуханием ALiBi, локально-глобальным окном (4+1) и токенами криvizны.

Для каждой тройки вейпоинтов вычисляется косинус угла поворота:

$$\cos \theta_k = \frac{\vec{v}_k \cdot \vec{v}_{k+1}}{\|\vec{v}_k\| \|\vec{v}_{k+1}\|}, \quad \vec{v}_k = w_{k+1} - w_k. \quad (6)$$

Это позволяет сети заранее «видеть» резкий поворот на 90 градусов и сбавлять скорость захода.

2. **Затухание внимания (ALiBi Distance-Decay):** В матрицу внимания вводится линейный штраф за удаленность:

$$\text{Attn}(Q, K) = \text{Softmax} \left(\frac{QK^\top}{\sqrt{d}} - \lambda m_h \cdot \mathbf{d} \right), \quad (7)$$

где $\mathbf{d} = [0, 1, \dots, K-1]$, $\lambda = 0.4$, $m_h = \frac{h}{H}$. Это гарантирует, что 80% энергии внимания концентрируется на ближайших маневрах, предотвращая опасную «утечку сквозь стены».

3. **Локально-глобальное окно (4+1):** Внимание ограничивается 4 ближайшими точками + 1 терминальным токеном цели $[B(g)]$, что отсекает шум от далеких поворотов.
4. **Вспомогательный лосс локализации:** $\mathcal{L}_{aux} = \|\hat{z}_{seq} - z_{single_wp}^*\|^2$ (вес 0.2), стабилизирующий ранние этапы обучения.

6 Экспериментальные результаты и анализ поведения

Для всесторонней оценки разработанных методов было проведено тестирование в средах `antmaze-medium-navigate-v0` и `antmaze-large-navigate-v0` на 20 независимых случайных сидах (10 эпизодов на задачу, 5 задач, суммарно 1000 эпизодов на каждый метод).

6.1 Анализ результатов на Medium лабиринте

В компактном лабиринте Medium коридоры сравнительно короткие, и плотность поворотов высока. В этих условиях наивысшую точность продемонстрировал `Single WP Translator` (94.0%), поскольку локальный адаптивный гейт идеально отрабатывает одиночные резкие повороты. `Distilled JAX` показал лучший баланс скорости и качества (86.0% при латентности всего 0.18 мс).

6.2 Триумф Sequence Reasoning на Large лабиринте

Картина кардинально меняется при переходе к лабиринту Large, где длина коридоров увеличивается втрое,

а число последовательных поворотов достигает 8–12.

На лабиринте Large базовый Single-Intention контроллер полностью деградирует (успешность падает до 24%), а Single WP опускается до 44%. В то же время **Enhanced Sequence Attention** достигает **66.4%** (на лучших сидах до **70.0%**) благодаря трем физическим эффектам:

1. **Снятие «близорукости» на длинных перегонах:** Одноточечный транслятор ориентируется строго на ближайшую точку. На высокой скорости робот не успевает погасить боковую инерцию и врежется во внутренний угол стены при входе в следующий поворот. Sequence Attention видит цепочку из 4 точек и угол кривизны $\cos \theta_k$, заблаговременно формируя плавную дугу захода.
2. **Устранение паразитного внимания через ALiBi:** Без штрафа за расстояние трансформер «видел» далекие терминальные участки за стеной и пытался спрямить путь. Линейный штраф ALiBi заставляет сеть отдавать 80% веса непосредственному повороту, сохраняя глобальную ориентацию в хвосте последовательности.
3. **Устойчивость к стохастике MuJoCo:** Скользящее окно упреждения исключает микроостановки, сохраняя равномерную походку (Gait cycle) муравья без опрокидываний.

7 Обсуждение, перспективы развития и выводы

7.1 Что еще можно было бы попробовать?

В ходе исследования сформировался ряд перспективных идей, которые могут быть реализованы в качестве дальнейшего развития проекта:

1. **Динамическая кластеризация ориентиров (Voronoi Landmark Pruning):** Сейчас ориентиры сэмпляются равномерно по буферу. Использование диаграмм Вороного и фильтрации по топологической кривизне позволит сократить число вершин с $N = 2000$ до $N = 300$ без потери разрешающей способности, ускоряя построение графа в 40 раз.
2. **Адаптивный температурный контроллер выхода из стагнации:** Вместо дискретного счетчика застреваний можно внедрить дифференцируемый блок оценки кинематического затухания $\Delta v = \|v_t - \bar{v}\|$, плавно модулирующий энтропию действий $\tau_t = \sigma(\alpha \cdot (v_{\text{crit}} - v_t))$ при контакте со стеной.
3. **Сквозная дистилляция траекторий в замкнутом цикле (Closed-Loop Rollout Distillation):**

Обучение трансформера последовательностей не на статических срезах (s, W) , а через сквозное дифференцирование развернутых траекторий во времени (BPTT) с использованием дифференцируемых физических движков (Brax / MuJoCo XLA).

4. **Ансамблирование окон внимания для сверхбольших лабиринтов (AntMaze Giant):** Комбинация короткого окна ($W = 4$) для прохождения узких створов и длинного окна ($W = 16$) для открытых залов.

7.2 Общие выводы

В данной работе был пройден полный путь исследования и разработки методов многошагового планирования поверх замороженных Forward-Backward представлений в задаче навигации сложного многосуставного робота:

- Показано, почему наивные методы деления пути пополам (Recursive Bisection) ломаются из-за топологической невыпуклости лабиринтов и диффузного просачивания оценок достижимости сквозь стены.
- Реализован надежный топологический планировщик Search on the Buffer с квази-метрической стоимостью ребер и непрерывным скользящим упреждением (Sliding Lookahead), решивший проблему застревания на базовом уровне.
- Предложена и обучена архитектура амортизированной нейросетевой дистилляции в $O(1)$ (Distilled JAX Gated-Attention) с 4-компонентным физико-ориентированным лоссом, обеспечившая ускорение инференса в 5 раз при росте успешности до ****86.0%**** на лабиринте Medium.
- Разработан SOTA-метод **Enhanced Sequence Waypoint Attention Transformer**, объединивший топологический граф Дейкстры, затухание внимания ALiBi, токены кривизны и локально-глобальное окно упреждения, что обеспечило уверенную победу над бейзлайном на лабиринте Large (****66.4%** против 24.0%**).

Полученные результаты наглядно доказывают, что правильное рассуждение о последовательностях интенций в сочетании с физико-согласованными механизмами внимания позволяет эффективно решать длинноразрешенные задачи zero-shot навигации без привлечения вычислительно тяжелых генеративных моделей мира.

Список литературы

- [1] Stojanovic M., Proutiere A. Switching Successor Measures for Hierarchical Zero-Shot Reinforcement Learning // arXiv preprint arXiv:2605.13207. — 2026.
- [2] Touati A., Ollivier Y. Learning One Representation to Optimize All Rewards // Advances in Neural Information Processing Systems (NeurIPS). — 2021. — Vol. 34. — P. 13–23.
- [3] Touati A., Rapin J., Ollivier Y. Does Zero-Shot Reinforcement Learning Exist? // International Conference on Machine Learning (ICML). — 2023. — P. 34500–34520.
- [4] Barreto A., Dabney W., Munos R., et al. Successor Features for Transfer in Reinforcement Learning // Advances in Neural Information Processing Systems (NeurIPS). — 2017. — Vol. 30.
- [5] Eysenbach B., Salakhutdinov R., Levine S. Search on the Replay Buffer: Bridging Planning and Reinforcement Learning // Advances in Neural Information Processing Systems (NeurIPS). — 2019. — Vol. 32.
- [6] Park S., Rhinehart N., Levine S. OGBench: Benchmarking Offline Goal-Conditioned Reinforcement Learning // International Conference on Learning Representations (ICLR). — 2025.
- [7] Press O., Smith N. A., Lewis M. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation // arXiv preprint arXiv:2108.12409. — 2021.

Таблица 1: Сравнение архитектур дистилляции на Medium

Архитектура	Парам.	Cos-Sim	SR (%)	Время
Standard MLP	312K	0.891	78.0%	0.12 мс
Dense-ECA	458K	0.914	82.0%	0.16 мс
ResNet-ECA	524K	0.908	80.0%	0.15 мс
Gated Cross-Attn	680K	0.942	86.0%	0.18 мс

Таблица 2: Сводная сравнительная таблица всех исследованных методов на AntMaze Medium и Large (20 сидов, 1000 эпизодов)

Метод планирования	AntMaze Medium			AntMaze Large		
	SR (%)	Шаги	Время (мс)	SR (%)	Task 01 (%)	Task 05 (%)
1. Single-Intention Baseline	76.0 ± 4.2	235	0.94	24.0 ± 3.5	10.0%	20.0%
2. Recursive Bisection	74.2 ± 5.1	260	1.45	32.0 ± 4.8	20.0%	30.0%
3. Dijkstra Teacher (<code>high_actor</code>)	78.0 ± 3.8	210	0.86	48.0 ± 4.2	50.0%	50.0%
4. Distilled JAX GatedAttn [$O(1)$]	86.0 ± 2.9	184	0.18	50.0 ± 3.9	60.0%	50.0%
5. Dijkstra + Single WP Translator	94.0 ± 2.1	165	0.56	44.0 ± 4.5	40.0%	40.0%
6. Dijkstra + Enhanced Sequence Attn	88.0 ± 3.0	172	0.82	66.4 ± 3.2	90.0%	70.0%